

159.102 Notes

Section 7 : Classes, Objects and Pointers

Classes and Objects – a reminder and overview

159.101 introduced programming with classes and objects. This section is a reminder of what was introduced in 159.101 and then goes on to some new concepts.

A **class** is a collection of **data** items and **methods** (functions). Usually the data in the classes is part of a logical collection – sometimes called a **record**. For example, a student record may consist of student name, degree, completed courses, etc.

Methods are functions defined within the class that manipulate the data of the class. The methods serve two purposes:

- a) they make main shorter and easier to read
- b) they make the classes portable (or stand-alone) so they can be placed in a library

Data and methods inside a class can be defined to be public or private. **Private** data and methods can only be used within the class itself – they may not be used by main (or any other function). **Public** data and methods can be used anywhere in the program (inside the class, main, other functions, etc).

You can decide which data and which methods are private and which are public. We will follow a widely-used convention which is:

- make all data private
- make all methods public
- write “set” and “get” methods as required

The reason for this is that it enables classes to become portable (i.e. can be held in a library and used by many different programs). This would not be possible if the main program had to know about the data in order to use the class.

It is very common to see “**set**” and “**get**” methods inside a class. The purpose of such methods is to keep the class data private. A “set” method takes a value (usually as a parameter) and loads it into the corresponding variable in the class. A “get” method returns the value of a class variable back to main (or another function outside of the class). The definitions of very short methods are often written on one line inside the class.

We can define several classes in one program and classes can make use of other classes. We can also set up arrays (or other data structures) of classes.

The Timetable Program – a case study of programming with classes

As a reminder of how to use classes and objects in a program, let us look at the timetable program.

What this program does

There is one student who is studying several courses. The timetable details for each course are loaded. The student’s timetable is checked to see if there are any timetable clashes, i.e. courses that are taught at the same time.

Simplifications in the program

It only works for one student – you could modify it to work for several students.

A timetable session has a day, a time and a room – it is assumed that every session is 1 hour.

A time is an integer representing the hour, e.g. 11 means 11am and 13 means 1pm.

A course has a code (e.g. 159101), one session and a note if it is a core course or not.

Real courses actually have other data, e.g. semester, lecturer name, online or not, etc.

Most courses actually have several sessions – you could modify it to allow this.

A student has a name and a set of courses – a course may actually have more data than this.

The program only checks for clashes, it could check other things, e.g. room bookings.

No data validation is done, it is assumed that all data is entered correctly.

In a real program, data for courses would probably be loaded from a file of course information.

The program

The program has three classes:

- session – this holds the data for one event in the timetable at a particular time.
- course – this holds the course code, the core indicator (not actually used) and the session. The course class uses the session class.
- student – this holds the student name and a number of courses. The student class uses the course class.

```
#include <iostream>
#include <cstdio> // to use getchar
using namespace std;

class session_class {
private:
    // a session in the timetable
    string day; // e.g. "Thursday"
    int hour;   // 11 = 11am, 13 = 1pm, etc
    string room; // e.g. "SNW300"
public:
    void loaddata();
    bool sametime(session_class other);
};

class course_class {
private:
    int code; // e.g. 159101
    bool core; // true if a core course, otherwise false
    // only one session - maybe there should be more sessions?
    session_class session;
public:
    void loaddata();
    int getcode() { return code; }
    bool sametime(course_class other);
};

class student_class {
private:
    string name; // name of the student
    int num;     // number of courses
    course_class courses[30];
public:
```

```

    void loadname();
    void loadcourses();
    void checkclashes();
};

// assume one student for now - then modify for many students
student_class student;

int main() {
    cout << "Check a student timetable for clashes\n";
    student.loadname();
    student.loadcourses();
    cout << "\nThis student has the following clashes:\n";
    student.checkclashes();
}

//----- methods for the session_class -----
void session_class::loaddata() {
    cout << "Enter the day (e.g. Thursday): ";
    cin >> day;
    cout << "Enter the hour (e.g. 11 = 11am and 13 = 1pm): ";
    cin >> hour;
}

bool session_class::sametime(session_class other) {
    if ((day == other.day) && (hour == other.hour)) {
        return true;
    }
    return false;
}

//----- methods for the course_class -----
void course_class::loaddata() {
    cout << "Enter the course code (e.g. 159101): ";
    cin >> code;
    session.loaddata();
}

bool course_class::sametime(course_class other) {
    // return true if both courses have a session at the same time
    if (session.sametime(other.session) == true) {
        return true;
    }
    return false;
}

//----- methods for the student_class -----
void student_class::loadname() {
    cout << "Enter name of student ";
    getline(cin, name);
}

void student_class::loadcourses() {
    int i;
    cout << "How many courses is this student doing? ";

```

```

    cin >> num;
    getchar(); // flush the input buffer
    for (i = 0; i < num; i++) {
        courses[i].loaddata();
    }
}

void student_class::checkclashes() {
// check for timetable clashes across the courses
// WARNING - this method could be improved
    int code1, code2, i, k;
    for (i = 0; i < num; i++) {
        for (k = 0; k < num; k++) {
            if (courses[i].sametime(courses[k]) == true) {
                code1 = courses[i].getcode();
                code2 = courses[k].getcode();
                cout << "Clash - " << code1 << " against " << code2 << endl;
            }
        }
    }
}
}

```

Important points about this program:

- there are three classes and two of the classes use other classes
- the method `getcode()` in class `course_class` is a short method defined on one line
- some data (e.g. room and core) are not used. You could update the program to use them...
- the program checks for clashes but it could do other things, e.g. check for core courses
- the program does not work very well – what is the problem with the results?

Note that both `session_class` and `course_class` have a method called `sametime`. Both these methods essentially do the same thing, i.e. they return true if two sessions are at the same time, otherwise they return false. This is an interesting situation where using methods causes duplication of code, whereas one of the main reasons for functions is to reduce the duplication of code. The problem is caused by the conventions about private class data.

So we have a choice – stick to the convention that all class data is private, or makes some class data public in order to reduce situations such as this. Difficult to say which is better and it often depends on the context of what you are doing.

The program is arranged to achieve two main objectives:

- a) main is very short and (hopefully) easy to read. All details are off-loaded into the classes.
- b) the classes are all portable. It would be easy to use them in other programs.

Portable classes

Classes should be portable. This enables them to be easily used in other programs and by other programmers. A well-constructed class means that you can change the data and the method code in a class without affecting the programs that use the class. This is particularly important if other programmers are using your class in their programs. They may not be too happy about having to change their program each time you modify the class (although sometimes a change to the program cannot be avoided. Reasons for changing data inside a class can include:

- increasing the number of variables in the class
- adding new methods to a class (but avoid changing the old methods)

- changing the data type of a variable to make it more efficient or more versatile
- changing the data structure used inside a class (e.g. changing from an array to a vector)

Example of changing a class – store the course code as a string

Assume that we have decided that we need to do some checks for invalid data. For this reason, we want to change the course code to be stored as a string. We would then change the `course_class` class to look like this (new code is in bold):

```
class course_class {
private:
    string code;    // e.g. 159101
    bool core;    // true if a core course, otherwise false
    // only one session - maybe there should be more sessions?
    session_class session;
public:
    void loaddata();
    int getcode() { return code; }
    bool sametime(course_class other);
};
```

Note how the public sections of the class remain unchanged. This means that any program using the class is not affected by the change. Note that the short one-line method `getcode` has more lines of code added, so it is no longer defined as a one-line method. The methods for the `course_class` would now be changed as follows (note that these methods are not yet complete):

```
//----- methods for the course_class -----
void course_class::loaddata() {
    cout << "Enter the course code (e.g. 159101): ";
    cin >> code;
// include code to check if the code is a valid course number
    session.loaddata();
}

int course_class::getcode() {
    int temp;
// include code to convert the string 'code' into the number 'temp'
    return temp;
}

bool course_class::sametime(course_class other) {
    // return true if both courses have a session at the same time
    if (session.sametime(other.session) == true) {
        return true;
    }
    return false;
}
```

These changes do not affect the programs (or functions) that use the class. They would continue functioning as if nothing has changed.

Using pointers with classes

Pointers can be used with classes in the same way they are used with ordinary variables.

```
#include <iostream>
using namespace std;

class date_class {
private:
    int day, month, year;
public:
    void setdate(int d, int m, int y);
    int getday() { return day; }
    int getmonth() { return month; }
    int getyear() { return year; }
    void displaydate();
};

date_class newyear;
date_class *ptr;

int main() {
    ptr = &newyear;
    ptr->setdate(1, 1, 2018);
    newyear.displaydate();
}

//----- methods for the date_class -----
void date_class::setdate(int d, int m, int y) {
    day = d;
    month = m;
    year = y;
}

void date_class::displaydate() {
    cout << day << "/" << month << "/" << year << endl;
}
```

Important points about this program:

- this program contains an object called newyear
- we set the pointer to hold the address of the object
- note the -> symbol to in the line `ptr->setdate(1, 1, 2018);` Read this symbol as saying: “carry out the method on the object pointed to by the pointer”.
- the program runs and displays the date stored in the object newyear which is 1/1/2018. Note that newyear was never loaded with this date. It was loaded indirectly via the pointer.

Pointers with no defined objects (use new to allocate memory)

Sometimes it is useful to have objects with no names at all. The objects are known only by the pointers that hold the relevant address. In this case, we set aside the required memory space by using new rather than defining an object.

```
#include <iostream>
using namespace std;
```

```

class date_class {
private:
    int day, month, year;
public:
    void setdate(int d, int m, int y);
    int getday() { return day; }
    int getmonth() { return month; }
    int getyear() { return year; }
    void displaydate();
};

date_class *ptr1, *ptr2;

int main() {
    ptr1 = new date_class; // sets aside enough memory for a date
    ptr2 = new date_class;
    ptr1->setdate(14, 7, 1789);
    ptr2->setdate(15, 8, 1947);
    ptr1->displaydate();
    ptr2->displaydate();
}

//----- methods for the date_class -----
void date_class::setdate(int d, int m, int y) {
    day = d;
    month = m;
    year = y;
}

void date_class::displaydate() {
    cout << day << "/" << month << "/" << year << endl;
}

```

Important points about this program:

- this program does not contain any defined objects – it uses only the pointers
- the instruction `new` is used to set aside enough memory to hold the designated class
- `you must use new` – the program cannot run otherwise as it has no memory to store the data

Using pointers as parameters to speed up programs

For those of us who worry about speed, it is faster to pass a pointer as a parameter than to pass an actual object. If an object is passed as a parameter then the whole object has to be copied to create the parameter object to be used inside the function. This can waste significant computer time (it depends on the size of the object). For this reason you will often come across functions where the parameters are pointers to objects and not actual objects.

Strings in C and C++

It is useful for C++ programmers to understand the difference between strings in C and strings in C++. Although C and C++ are regarded as two different languages, they are identical in most respects. This is good news for you because once you have learned C++ you can put in your CV that you also know C (well, most of C anyway). By the way, if you want to be sure you can actually program in C (as well as C++) have a look at this website: <https://www.learn-c.org/> You should know everything that is listed under “Learn the Basics”. One difference between C and C++ is their approach to strings.

Strings in C

A string in C is an array of characters (there is no “string” as such). The last character in the string is the special character `\0`. It is used to indicate “this is the end of the string”.

To define a string variable in C you would set up an array of characters, for example:

```
char str[60];
```

You could then load characters (one by one) into the string.

Problems with this approach:

1. There is a maximum length, e.g. in the above example `str` cannot hold more than 59 characters
2. It is an array and so you cannot use the code: `str = "Hello";`
3. You cannot compare strings, for example this would not work: `if (str1 == str2)`

Strings in C often use pointers, especially as parameters. If you see a C function like this:

```
void function(char *ptr);
```

It usually means that the parameter is a C string. The pointer is used to avoid copying the whole array in order to use the function.

Strings in C++

A string in C++ is a class defined like this:

```
class string {
private:
    a C-string consisting of an array of characters
    some other data items

public:
    int length(); // this returns the length of the string
    void push_back(char ch); // inserts ch at the end of the string
    c_str(); // returns a pointer to the C-string
    many other methods
};
```

We can then define a string object like this: `string str;` (and we are familiar with doing this)

Here is the website for the full definition of the string class in C++:

<http://www.cplusplus.com/reference/string/string/>

WARNING: looking at this website can cause severe shock!! The website is NOT part of our course.

Why bother with C strings?

There are often useful functions that have a parameter that is a C string (and not a C++ string). This may be for historical reasons, i.e. the function was used in C before C++ was invented. The following program demonstrates using a C++ string with a function that requires a C string parameter.

```
#include <iostream>
using namespace std;

int counting(char *ptr);

string str;

int main() {
    int total;
    char *strptr;
    str = "Good bye";
    strptr = (char*) str.c_str();
    total = counting(strptr);
    cout << "The string has " << total << " characters.\n";
}

int counting(char *ptr) {
    int i;
    i = 0;
    while (ptr[i] != '\0') {
        i++;
    }
    return i;
}
```

Important points about this program:

- the pointer `strptr` is loaded with the address of the C string inside the object `str`
- the function is a C function (not C++) that expects a C string parameter
- the function knows that the C string will contain `'\0'` after the other characters
- you don't need this function (you could use `str.length()`) but the point is to study how to pass a parameter to a function that expects a C string.

The fopen function in fstream

We have already used a function that expects a C string as a parameter, namely the `open` function in the library `fstream`. We have used lines of code such as:

```
datafile.open(filename.c_str(), fstream::out);
```

The first parameter required by the `open` function is a pointer to the name of the file stored as a C string. We stored the name of the file in a C++ string so we need to use `c_str()` to get the required pointer.